FSM-Controlled Smart Football Event Detection ASIC

Farzan Bhalara (202301248), Ashka Pathak (202301270)
Dhirubhai Ambani University
202301248@dau.ac.in, 202301270@dau.ac.in

***Abstract*—In this paper, we present a custom application-specific integrated circuit (ASIC) for smart football event detection using accelerometer and gyroscope inputs. Rather than building the system around a microcontroller or a machine-learning pipeline, we chose a fully deterministic digital architecture so that the behavior would stay easier to verify, interpret, and debug. The design is organized around a sensor-event extraction block** (event_detect)**, a compact finite-state-machine controller** (event_fsm)**, and a top-level integration module. We carried the implementation through a complete open-source flow that included RTL simulation, Yosys synthesis, and OpenLane floorplanning, placement, clock-tree preparation, routing, and signoff on the Sky130 platform. The functional waveforms confirm correct sequencing of** sample_tick**,** sample_done**,** process_done**,** detect_en**, and the 3-bit** fsm_state **during event qualification. Synthesis results show that** event_detect **dominates logic complexity with 10,915 cells, whereas** event_fsm **requires only 26 cells. Post-layout metrics report a 0.247 mm^2 die area, 9,369 synthesized logic cells, 29,936 total placed physical cells, and clean signoff with DRC = 0 and LVS = 0. Although post-layout timing remains negative (WNS = -2.77 ns), the project still demonstrates a complete hardware-only smart football sensing ASIC with manufacturable geometry and correct layout-versus-schematic connectivity.**

***Index Terms*—ASIC, smart sports sensing, event detection, finite-state machine, Yosys, OpenLane, Sky130**

I. Introduction

Smart sports equipment is steadily moving toward always-on sensing, faster event recognition, and more compact on-device intelligence. In football monitoring, accelerometer and gyroscope signals carry useful clues about impacts, kicks, ball motion, and handling events. The real challenge is turning those raw signatures into reliable decisions while still staying within tight limits on size, latency, and energy.

For this project, we deliberately took a pure digital ASIC route. That choice let us replace firmware execution and inference software with fixed-function hardware logic. Compared with a microcontroller-based solution, a dedicated chip can reduce decision latency, remove software scheduling uncertainty, and avoid the overhead of instruction fetches, memory activity, and firmware maintenance. Compared with a machine-learning-driven pipeline, a deterministic circuit is easier to interpret, has bounded execution time, and can be verified more directly, which is especially useful when the target event can be described using explicit sensing rules instead of large data-driven models [1], [2], [4].

With that motivation, we developed a pure-hardware smart football event detection system as a modular RTL design and pushed it through a full open-source ASIC flow [5]–[9]. Our contributions are fourfold. First, we implement a deterministic accelerometer–gyroscope event detector organized around event_detect, event_fsm, and top. Second, we validate the design behavior using implemented verification waveforms. Third, we carry the design through logic synthesis, floorplanning, placement, routing, and signoff using Yosys and OpenLane on Sky130. Finally, we analyze the post-layout results, including the clean DRC/LVS reports and the timing-closure limitations that point to the next round of optimization.

II. System Architecture

At a high level, the proposed smart football ASIC follows a staged sensing pipeline. Digital inputs derived from the accelerometer and gyroscope are first aligned by an interface layer, then reduced into event flags, sequenced by a finite-state controller, and finally converted into an output-valid indication. Figure 1 gives the overall picture. We kept this partition intentionally simple: the event-detection logic handles the larger combinational workload, while the FSM keeps the control path compact and predictable.

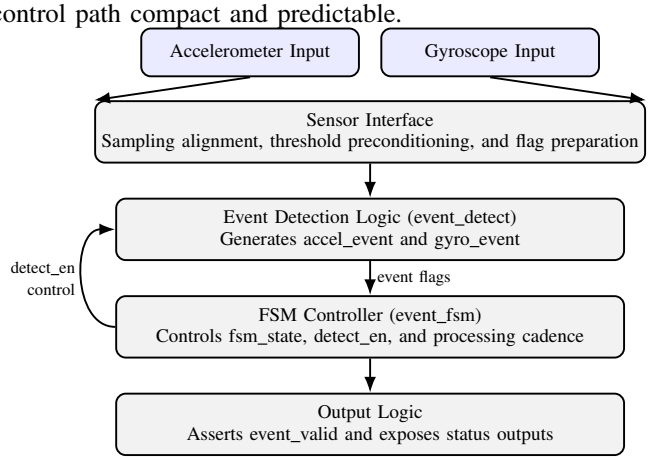

Fig. 1. Block diagram of the proposed smart football ASIC architecture.

In practice, the architecture behaves like a gated event qualifier. Sensor-related flags are computed first, but they matter only when the FSM opens the detection window. That split helped us during debugging because it separated one question—"did the sensors trigger?"—from another equally important one—"was the system ready to accept that trigger?" As the waveform results later show, event_valid does not

simply follow raw sensor activity; it becomes active only when the control sequence reaches the proper state.

III. RTL Design

Once the overall architecture was fixed, we organized the RTL into three modules using a familiar datapath-and-controller split. Keeping the modules separate made the simulations easier to read and, later, made the synthesis reports much more informative because we could quickly see which block was responsible for most of the area and logic depth.

A. event_detect *Module*

The event_detect block is clearly the heaviest logic block in the system. Its synthesis statistics report 10,925 wires, 11,270 wire bits, and 10,915 cells, which points to a wide datapath-style implementation. The gate mix is dominated by Boolean comparison and reduction logic, including 3,349 ANDNOT cells, 2,346 XOR cells, and 1,470 XNOR cells. This is exactly the kind of structure we would expect from a deterministic sensor-event evaluator that compares, masks, and combines multiple input conditions before deciding whether accelerometer or gyroscope events should be asserted.

B. event_fsm *Module*

By comparison, the event_fsm block is intentionally small. It uses only 28 wires, 32 wire bits, and 26 cells, including three sequential elements. For us, that was an encouraging result because it meant the sequencing and control logic stayed lightweight, leaving most of the implementation effort where it belonged: in the event-processing block. Together with the 3-bit state output seen in simulation, the small cell count suggests a compact set of encoded states implementing a deterministic acquisition and validation schedule.

C. Top-Level Integration

The top module ties the detector and controller together and exposes the full system interface. Its Yosys statistics report 137 wires, 291 wire bits, and 185 cells. The report also explicitly lists one event_detect instance and one event_fsm instance, which confirms that hierarchy is preserved at the top level. The remaining glue logic includes 35 flip-flops, 44 OR cells, 37 ANDNOT cells, and 29 XOR cells, providing the sequencing, gating, and output formatting needed around the two main submodules.

IV. Functional Verification

Before moving into physical implementation, we verified the RTL with representative stimulus to make sure the behavior stayed deterministic and the top-level sequencing remained correct across repeated sampling windows. These waveforms turned out to be some of the most useful debug artifacts in the project because they let us check both event timing and state progression early.

Figure 2 shows the full simulation interval. After the initial reset period, rst_n is deasserted and the design enters steady operation. Periodic sample_tick pulses establish a recurring cadence in which sample_done and process_done follow as downstream handshake signals. During the applied motion window, both accel_event and gyro_event assert, but event_valid becomes active only after the FSM reaches the qualifying detection phase. To us, this is an important result because it shows that the output is not just a raw threshold crossing; it is a controlled decision produced jointly by the datapath and the FSM.

Figure 3 gives a closer look at the transition ordering during one detection interval. The 3-bit fsm_state advances through $000 \rightarrow 001 \rightarrow 010 \rightarrow 011 \rightarrow 000$, indicating a multi-cycle state progression rather than a single-cycle pulse detector. Within this interval, sample_tick marks the observation boundary and sample_done confirms acquisition. Next, process_done marks completion of internal evaluation, and detect_en is asserted during the final qualification stage. Because accel_event and gyro_event remain stable while detect_en is pulsed, the detector validates motion only when both sensor evidence and controller timing line up. That sequence gave us confidence that the FSM gate was doing its job rather than accidentally latching a brief sensor pulse.

V. ASIC Design Flow

Once the functional behavior looked correct, we took the same RTL through a conventional digital ASIC flow. As summarized in Fig. 4, RTL coding and testbench simulation were followed by Yosys synthesis and then by OpenLane floorplanning, placement, clock-tree preparation, routing, and signoff checks [5]–[7], [9]. Using an open-source flow from end to end was especially useful for a student project because every stage produced reports we could inspect directly instead of treating the backend as a black box.

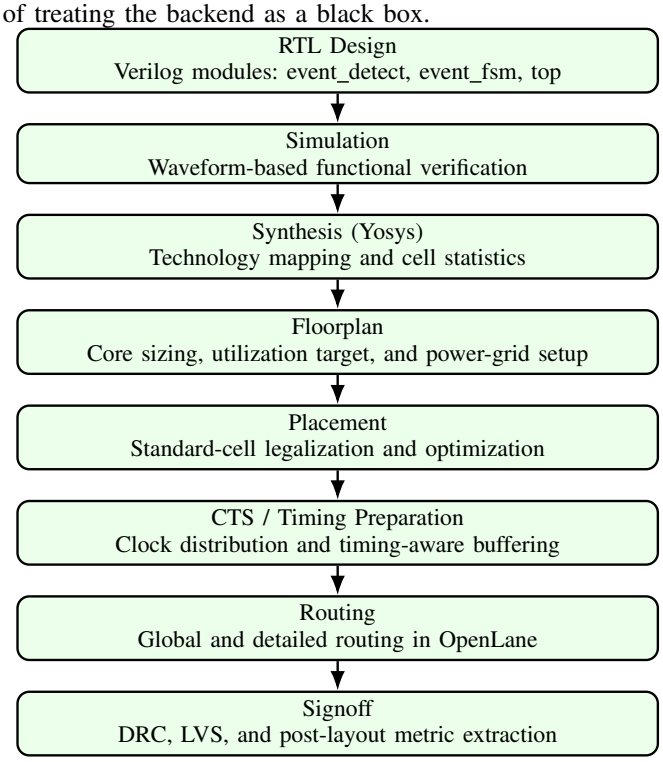

Fig. 4. Open-source ASIC design flow used in this work.

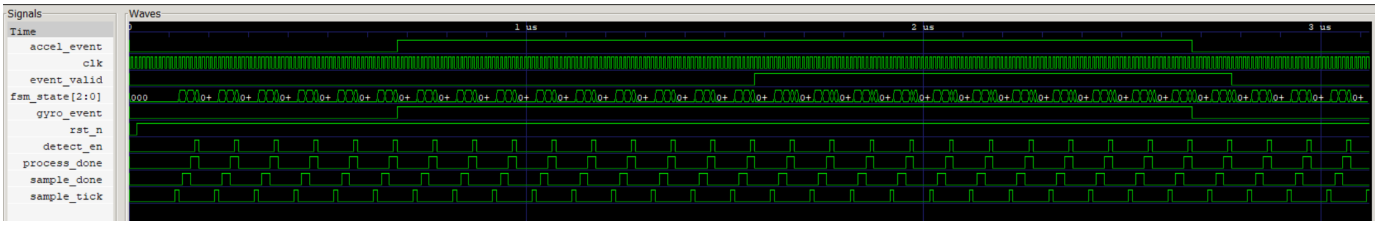

Fig. 2. Full functional waveform of the smart football detector.

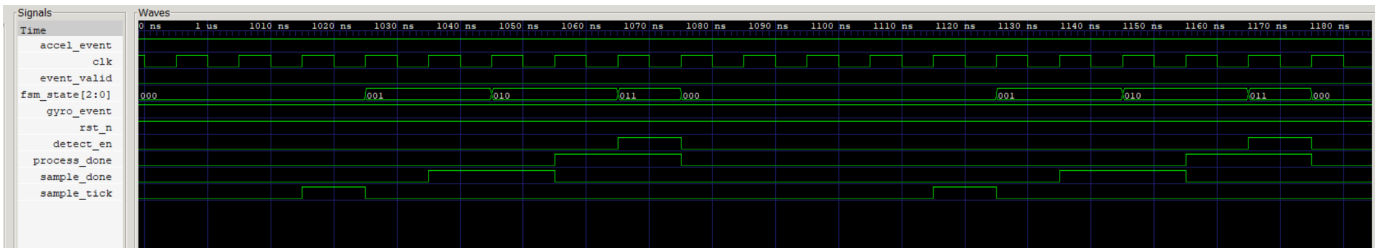

Fig. 3. Zoomed waveform for one event-detection interval.

```
5. Printing statistics.

=== $paramod$fb66d5c2c0bb977129f50798f4fd16f35a8b13e0\event_detect ===

   Number of wires:                10925
   Number of wire bits:            11270
   Number of public wires:            23
   Number of public wire bits:       368
   Number of memories:                 0
   Number of memory bits:              0
   Number of processes:                0
   Number of cells:                10915
     $_ANDNOT_                      3349
     $_AND_                          856
     $_DFFE_PN0P_                     11
     $_MUX_                           10
     $_NAND_                         350
     $_NOR_                          883
     $_NOT_                          385
     $_ORNOT_                        395
     $_OR_                           860
     $_XNOR_                        1470
     $_XOR_                         2346
```

Fig. 5. Yosys synthesis statistics for event_detect.

This flow matters because it exposes two different kinds of success. The waveform results tell us whether the logic behaves correctly. The backend results tell us whether that same logic can actually be placed, routed, and checked against fabrication and connectivity rules. In short, simulation tells us the idea works; physical implementation tells us the idea can survive a real chip flow.

VI. Synthesis Results

After that, the synthesis reports give a more concrete view of where the design effort is concentrated. The Yosys screenshots show the complexity and gate mix of each module, which makes it easier to see early on which blocks are likely to dominate area and timing.

Figure 5 makes it clear that event_detect is the computational center of the design. The block contains 10,915 cells, and about one third of them are ANDNOT cells, while XOR/XNOR structures account for another large share. This gate mix looks very much like the kind of Boolean comparison network we would expect in a rule-based event detector. In

```
=== event_fsm ===

   Number of wires:                28
   Number of wire bits:            32
   Number of public wires:         12
   Number of public wire bits:     16
   Number of memories:              0
   Number of memory bits:           0
   Number of processes:             0
   Number of cells:                26
     $_ANDNOT_                     11
     $_DFFE_PN0P_                   3
     $_NAND_                        2
     $_NOR_                         2
     $_NOT_                         1
     $_ORNOT_                       5
     $_OR_                          2
```

Fig. 6. Yosys synthesis statistics for event_fsm.

other words, the detector is not a simple threshold latch; it is a broad combinational decision network built for deterministic event extraction from sensor conditions.

By contrast, Fig. 6 shows that event_fsm uses only 26 cells, including three sequential elements and a small amount of surrounding logic. That compact implementation is desirable because it keeps the control overhead low while still enforcing a clean, staged decision sequence. It also confirms that most of the design complexity remains in the event logic, not in the controller.

Figure 7 shows 185 cells and 35 flip-flops at the top level, together with explicit instantiation of one event_detect block and one event_fsm block. This matches the hierarchy we intended from the start: a heavy event-processing block, a small FSM controller, and a relatively light wrapper. From an optimization point of view, the report also makes it clear

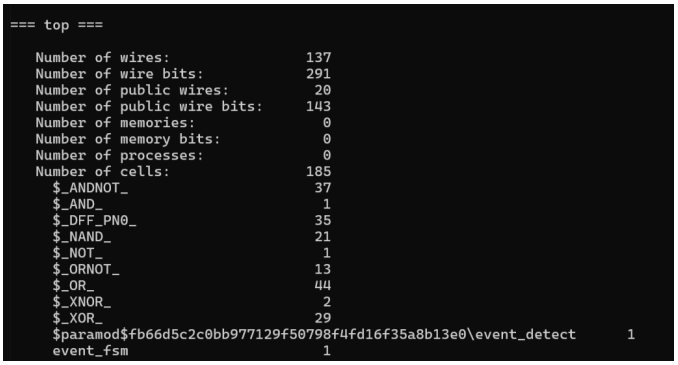

Fig. 7. Yosys synthesis statistics for the top module.

where future pipelining or logic balancing would matter most.

VII. Physical Design

After synthesis, we carried the design through floorplanning, placement, and routing in OpenLane [6]–[8]. The resulting layout shows that the smart football detector can indeed be implemented as a dense standard-cell ASIC on Sky130. It also helps explain why the remaining whitespace should not be read as wasted area; much of it reflects routing headroom and other implementation needs in a mixed control-plus-datapath design.

Figure 8 shows a conventional row-based digital implementation with dense standard-cell placement over most of the core. The visible horizontal rows and orthogonal routing patterns point to a cell-based design rather than one dominated by large macros or analog islands. The placement is not completely saturated, which is actually useful here because some whitespace is intentionally left for buffering, routing detours, tap insertion, and signoff cleanup. That reading is consistent with the post-layout metrics, which report 41.18% achieved utilization against a 40% target.

The same flow also produces the final routed GDS-II view discussed later in Fig. 12. That image complements the MAG snapshot by showing the full post-routing geometry and net distribution.

VIII. Physical Verification Results

Once routing is complete, the signoff reports tell us whether the layout is both manufacturable and logically equivalent to the intended netlist. For a student-scale ASIC effort, reaching clean physical verification feels like a major milestone because it means the project has moved beyond functional correctness and into physical consistency.

Figure 9 shows a DRC count of zero. In practical terms, that means no remaining geometry, spacing, or enclosure violations were reported at signoff, which is a basic requirement for a fabrication-ready layout. This was reassuring because it showed that the dense routing visible in the layout did not come at the cost of rule violations.

Figure 10 reports zero LVS errors and explicitly states that no net, device, pin, or property mismatches remain. That result matters because it shows that the fabricated geometry still corresponds to the intended logical design, not just that the layout is geometrically legal. Taken together, DRC = 0 and LVS = 0 give us confidence that the design is clean from both geometry and connectivity perspectives.

TABLE I
Post-layout metrics extracted from the OpenLane report.

Metric	Value
Flow status	flow failed
Die area	0.2471 mm^2
Core area	229,900.49 μm^2
Core utilization target / achieved	40% / 41.18%
Synthesized cell count	9,369
Total cells after physical insertion	29,936
Wire length (reported)	305,212
Via count	68,115
Worst negative slack (WNS)	-2.77 ns
Total negative slack (TNS)	-23.61 ns
Critical path	10.76 ns
Suggested clock period / frequency	12.71 ns / 78.68 MHz
Peak memory usage	996.11 MB

IX. Post-Layout Metrics

With signoff complete, the OpenLane metrics report gives the numerical side of the story: area, utilization, wiring complexity, and timing. It connects what we see in the layout screenshots with the actual outcome of the run.

Figure 11 was transcribed into the quantitative summary in Table I. The values show a moderate-size core with healthy placement utilization and a large jump from synthesized logical cells to total physical cells. That jump is expected because post-layout accounting includes fillers, well taps, decaps, and other support cells in addition to synthesized logic. So the difference between 9,369 synthesized cells and 29,936 total physical cells looks consistent with a complete physical implementation rather than a synthesis-only result.

The metrics also say something useful about routing. Reported layer usage is concentrated in the middle of the stack, with about 34.79% and 39.57% occupancy on routing layers 2 and 3, while upper-layer demand stays relatively small. That suggests a routing problem that is manageable rather than dominated by severe long-distance interconnect demand. Timing closure, however, is still unfinished: the negative WNS and TNS values point to unresolved setup violations. Since DRC and LVS are already clean, the remaining problem is performance closure, not manufacturability.

For completeness, Fig. 12 shows the final routed GDS-II view of the implemented chip. It complements the MAG layout by exposing the full post-routing geometry and net distribution.

Figure 12 shows a dense interconnect fabric spread across nearly the entire active core. The visible routing resources and peripheral labels make it clear that the design has been assembled as a full chip-level implementation rather than just a placed netlist snapshot. Together with Fig. 8, it confirms successful physical realization of the smart football detector from cell placement through final routed geometry.

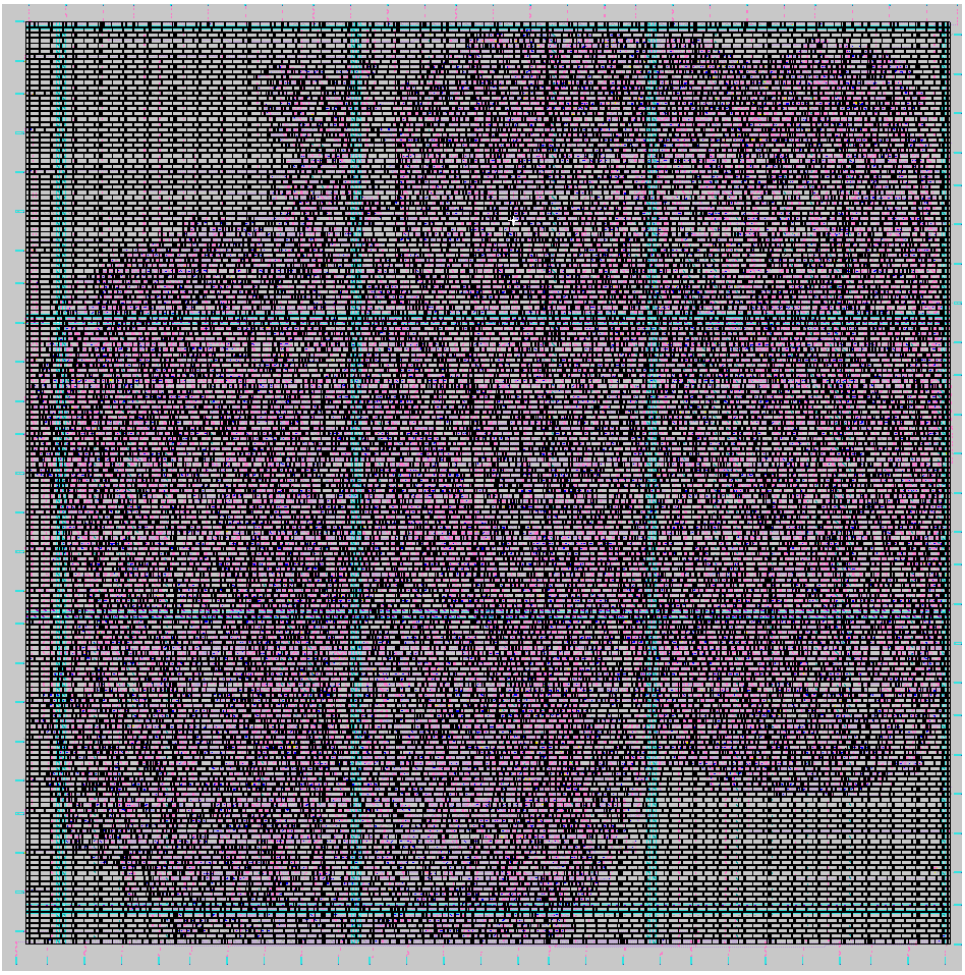

Fig. 8. OpenLane MAG view of the proposed smart football ASIC layout.

```
OpenLane Container (ff5509f):/openlane/designs/smart_football/runs/RUN_2026.04.21_18.56.47/reports/si
gnoff$ cat drc.rpt
top
----------------------------------------
[INFO]: COUNT: 0
[INFO]: Should be divided by 3 or 4
```

Fig. 9. Signoff DRC report showing zero violations.

```
OpenLane Container (ff5509f):/openlane/designs/smart_football/runs/RUN_2026.04.21_18.56.47/reports/si
gnoff$ cat 39-top.lvs.rpt
LVS reports no net, device, pin, or property mismatches.

Total errors = 0
```

Fig. 10. Signoff LVS report showing zero total errors.

X. Fabrication Path

Although this paper focuses on design and signoff, the final GDS-II layout in Fig. 12 is the point where the digital design connects to manufacturing. In a foundry-oriented flow, the handoff moves from RTL through OpenLane implementation to GDS-II export, and then on to CMOS fabrication and packaging. For our design, that means the logic we verified in simulation and signoff becomes the geometric template that could be manufactured in silicon [3], [4], [8].

Figure 13 summarizes the physical realization path once the routed layout has been finalized. Even though our work stops at design signoff, this view makes it easier to see how the verified GDS-II database would translate into a manufacturable chip.

1) Mask generation: The signoff-clean GDS-II file is translated into a set of photomasks, one for each process layer needed during manufacturing. At this stage, the polygons created during layout become the exact patterns that define active regions, gates, contacts, and metal interconnects.

```
OpenLane Container (ff5509f):/openlane/designs/smart_football/runs/RUN_2026.04.21_18.56.47/reports$ cat metri
cs.csv
design,design_name,config,flow_status,total_runtime,routed_runtime,(Cell/mm^2)/Core_Util,DIEAREA_mm^2,CellPer
_mm^2,OpenDP_Util,Final_Util,Peak_Memory_Usage_MB,synth_cell_count,tritonRoute_violations,Short_violations,Me
tSpc_violations,OffGrid_violations,MinHole_violations,Other_violations,Magic_violations,pin_antenna_violation
s,net_antenna_violations,lvs_total_errors,cvc_total_errors,klayout_violations,wire_length,vias,wns,pl_wns,opt
imized_wns,fastroute_wns,spef_wns,tns,pl_tns,optimized_tns,fastroute_tns,spef_tns,HPWL,routing_layer1_pct,rou
ting_layer2_pct,routing_layer3_pct,routing_layer4_pct,routing_layer5_pct,routing_layer6_pct,wires_count,wire_
bits,public_wires_count,public_wire_bits,memories_count,memory_bits,processes_count,cells_pre_abc,AND,DFF,NAN
D,NOR,OR,XOR,XNOR,MUX,inputs,outputs,level,DecapCells,WelltapCells,DiodeCells,FillCells,NonPhysCells,TotalCel
ls,CoreArea_um^2,power_slowest_internal_uW,power_slowest_switching_uW,power_slowest_leakage_uW,power_typical_
internal_uW,power_typical_switching_uW,power_typical_leakage_uW,power_fastest_internal_uW,power_fastest_switc
hing_uW,power_fastest_leakage_uW,critical_path_ns,suggested_clock_period,suggested_clock_frequency,CLOCK_PERI
OD,FP_ASPECT_RATIO,FP_CORE_UTIL,FP_PDN_HPITCH,FP_PDN_VPITCH,GRT_ADJUSTMENT,GRT_REPAIR_ANTENNAS,MAX_FANOUT_CON
STRAINT,PL_TARGET_DENSITY,RUN_HEURISTIC_DIODE_INSERTION,STD_CELL_LIBRARY,SYNTH_STRATEGY
/openlane/designs/smart_football,top,RUN_2026.04.21_18.56.47,flow failed,0h9m37s0ms,0h7m22s0ms,96893.76972541
591,0.247074709425,38757.507890166366,41.18,-1,996.11,9369,0,0,0,0,0,0,0,4,4,0,-1,-1,305212,68115,-2.77,-1,-1
,-1,-2.71,-23.61,-1,-1,-1,-23.5,252059204.0,0.0,34.79,39.57,3.99,10.1,0.0,10855,11358,53,525,0,0,0,10889,868,
35,323,897,913,2346,1440,10,145,49,37,12524,3293,4,4539,9576,29936,229900.4928,-1,-1,-1,0.0116,0.0151,5.57e-0
8,-1,-1,-1,10.76,12.71,78.67820613690007,10,1,40,153.18,153.6,0.3,1,10,0.5,0,sky130_fd_sc_hd,AREA 0
```

Fig. 11. Post-layout metrics summary used for quantitative analysis.

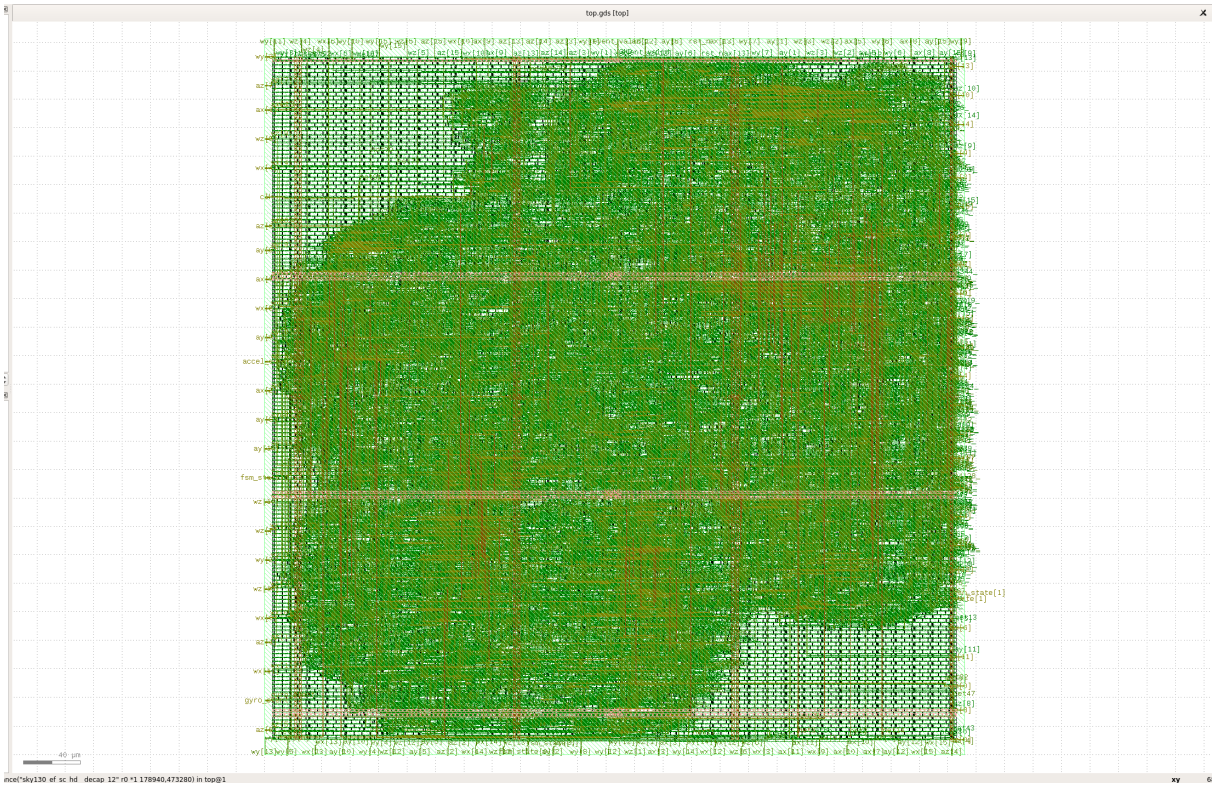

Fig. 12. Final routed GDS-II view of the implemented chip.

2) Photolithography: A silicon wafer is coated with photoresist, exposed to ultraviolet light through the appropriate mask, and then developed to reveal the desired pattern. This step repeatedly transfers the layout geometry from the mask set onto the wafer layer by layer.

3) Oxidation: Silicon dioxide is thermally grown or deposited to create insulating and protective layers on the wafer surface. In CMOS fabrication, oxide plays several roles, including gate insulation, isolation, and process surface preparation.

4) Ion implantation: Controlled doses of dopant ions are introduced into selected wafer regions to create n-type and p-type areas for NMOS and PMOS devices. The implanted profiles set up wells, source/drain regions, and threshold-adjustment regions that determine transistor behavior.

5) Etching: After lithography defines the protected and exposed areas, chemical or plasma etching removes unwanted oxide, polysilicon, or metal. This step physically sculpts the transistor features and interconnect openings according to the layout pattern.

6) Thin-film deposition: Additional material layers such as polysilicon, dielectric films, and barrier layers are deposited across the wafer. Deposition provides the raw material needed for later patterning steps and helps build up the multilayer stack used in modern CMOS processes.

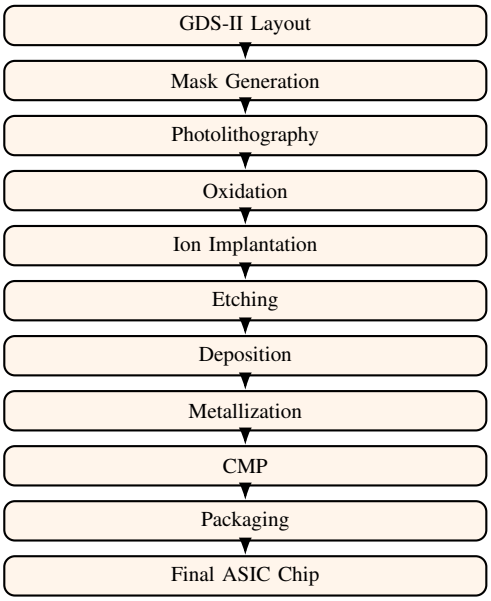

Fig. 13. CMOS fabrication flow for the proposed smart football ASIC.

7) Metallization: Metal layers and vias are formed to connect the transistors into the circuit described by the netlist and represented in the routed layout. For this smart football ASIC, the wires seen in Fig. 12 correspond to the interconnect patterns that would be realized during this stage.

8) CMP: Chemical-mechanical planarization smooths the wafer surface after deposition and metallization so that subsequent layers can be patterned accurately. Without planarization, topography would accumulate across the stack and make lithography and alignment increasingly difficult.

9) Packaging: Once wafer fabrication and testing are complete, individual dies are separated, bonded, and enclosed in a package that exposes the chip pins to the outside world. Packaging protects the silicon and makes the final ASIC practical to integrate into a sensing platform such as the proposed smart football system.

In short, the fabrication path can be read as one continuous chain: RTL design defines the behavior, OpenLane turns that behavior into a manufacturable layout, GDS-II captures the final geometry, and the CMOS process turns that geometry into a physical silicon chip. That is why Fig. 12 is more than a backend screenshot—it is the manufacturing-ready description of the ASIC.

XI. Discussion and Limitations

From our perspective, the main contribution of this work is showing that a fully synthesizable, hardware-only smart football event detection system can be taken through a complete open-source ASIC flow. The RTL stays modular, the controller stays compact, the event-processing block is easy to identify, and the backend artifacts correspond to a routed, signoff-clean implementation rather than a schematic-only exercise.

A second strength is the clarity of the architectural split. The synthesis data show a deliberate separation between a heavy detection datapath and a very small FSM controller. That separation is useful in deterministic embedded sensing because it keeps the sequencing behavior easy to follow while focusing optimization effort where it matters most: inside event_detect. For us, that made debugging, verification, and future physical optimization much more manageable.

The main limitation is still timing closure. The overall metrics file marks the run as flow failed, and the negative WNS/TNS values point to unresolved setup violations. Because DRC and LVS are both zero, we interpret this as a closure problem rather than a physical-rule failure. The large combinational footprint of event_detect suggests that logic depth, high-fanout control distribution, or both are likely contributors. The most direct next steps are to pipeline long Boolean paths, balance combinational trees, buffer high-fanout nets more aggressively, and retune floorplan density or placement constraints.

XII. Conclusion

In this paper, we presented a complete ASIC implementation of a smart football event detection system based on accelerometer and gyroscope inputs. By using deterministic digital logic and an FSM-based controller instead of a microcontroller or a machine-learning engine, the design targets low-latency, always-on embedded sports sensing. Functional waveforms confirmed correct event sequencing, Yosys reports showed the logic composition of each module, OpenLane produced a dense standard-cell layout, and signoff established DRC = 0 and LVS = 0.

The post-layout metrics show a routed, signoff-clean implementation with 0.2471 mm^2 die area and 29,936 total physical cells, which supports the feasibility of the architecture as a real chip. Although timing closure is still unfinished, the project validates the full path from RTL to manufacturable layout and finally to a fabrication-ready GDS-II representation. In future work, we would focus on reducing combinational depth, improving slack through targeted restructuring, and evaluating the detector across additional football-specific event classes and, eventually, silicon measurements.

References

[1] N. H. E. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Boston, MA, USA: Pearson/Addison-Wesley, 2011.

[2] J. M. Rabaey, A. Chandrakasan, and B. Nikolić, *Digital Integrated Circuits: A Design Perspective*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2003.

[3] J. D. Plummer, M. D. Deal, and P. B. Griffin, *Silicon VLSI Technology: Fundamentals, Practice, and Modeling*. Upper Saddle River, NJ, USA: Prentice Hall, 2000.

[4] S. M. Sze and K. K. Ng, *Physics of Semiconductor Devices*, 3rd ed. Hoboken, NJ, USA: Wiley, 2007.

[5] T. Ajayi *et al.*, "Toward an open-source digital flow: First learnings from the OpenROAD project," in *Proc. 56th ACM/IEEE Design Automation Conf. (DAC)*, Las Vegas, NV, USA, 2019, Art. no. 76.

[6] Efabless Corp. and contributors, "OpenLane documentation," 2026. [Online]. Available: https://openlane.readthedocs.io/. Accessed: Apr. 21, 2026.

[7] The OpenROAD Project, "OpenROAD documentation," 2026. [Online]. Available: https://openroad.readthedocs.io/. Accessed: Apr. 21, 2026.

[8] SkyWater PDK Authors, "SkyWater SKY130 PDK documentation," 2026. [Online]. Available: https://skywater-pdk.readthedocs.io/en/main/. Accessed: Apr. 21, 2026.

[9] C. Wolf, “Yosys Open SYnthesis Suite,” 2026. [Online]. Available: https://yosyshq.net/yosys/. Accessed: Apr. 21, 2026.